

Long Complex Resource Pack: Sumon, TOMAA, and ChatbotDev Migration Test

Purpose of this document

This 10-page resource is designed as a dense ingestion test for a new FastAPI-based chatbot backend. It intentionally mixes narrative business context, technical architecture, API requirements, data models, platform vocabulary, edge-case wording, timelines, and retrieval questions. The goal is not to be pretty marketing material. The goal is to give a bot a long, complex PDF that contains many related concepts and asks it to preserve identity, project intent, and implementation detail during retrieval.

Primary identity context

The user identity for the technical side is Sumon. Sumon is the programmer/developer persona working across projects including chatbotDev, myguru_dev, tomaadev, sumoncse07, and tefapi dev. In this context, Sumon is migrating a chatbot system from a Laravel API backend to a FastAPI backend while keeping the React TypeScript UI behavior close to the previous working version.

Shared-account context matters. Jimmy is the business owner and strategy side. Sumon is the developer side. When a message is technical, especially about code, APIs, React components, FastAPI, Laravel, routing, database structure, RAG behavior, embeddings, or chatbot debugging, the assistant should assume the speaker is Sumon unless the message explicitly says Jimmy.

Project summary

- chatbotDev is the current narrow project context for chatbot API and UI development.
- The old baseline used Laravel API plus React TypeScript UI.
- The new backend direction is FastAPI while preserving the same UI logic and user-facing flow as much as possible.
- The bot must support long context resources, resource upload, retrieval, guided flow, conversation state, and testing with complex PDFs.
- TOMAA is a major business/product context that must be understood because the chatbot may be tested against TOMAA-related content.

Test expectation

A good chatbot should answer specific questions from this PDF without inventing details. It should know that TOMAA is not the automotive company at toma.com. It should know TOMAA means automation around omni-channel content creation, social distribution, and human-in-the-loop approval. It should know Sumon is focused on implementation, not broad marketing theory.

Page 2 - Sumon Technical Profile and Working Style

Sumon is the developer persona responsible for turning business requirements into working software. He works mostly by sharing full code files, screenshots, route files, component files, controller code, and error logs. He prefers direct answers and complete replacement code when possible. When a technical issue is being debugged, partial snippets can create confusion; full files or clear SQL commands are preferred.

Developer preferences

- Answer like a senior software engineer.
- Be direct and concise. Do not over-explain unless asked.
- For coding tasks, provide full copy-paste replacement code when practical.
- Preserve existing functionality unless explicitly asked to change behavior.
- Prefer scalable, modular architecture over quick hacks.
- Use good patterns for FastAPI, React, TypeScript, PostgreSQL, JWT or token authentication, file upload handling, and RAG retrieval.
- When modifying a system, briefly explain what changed after the code.

Common working pattern

Sumon often begins by saying that he will share code and asks the assistant to wait or review what is missing. He may upload backend routes, frontend source, or zipped code. The assistant should compare old Laravel behavior against the new FastAPI behavior and identify missing endpoints, missing request/response fields, missing auth assumptions, missing guided-flow logic, missing UI state handling, and broken schema alignment.

Area	Old baseline	New target	Risk
Backend framework	Laravel controllers and routes	FastAPI routers, services, schemas	Endpoint parity can break if paths or JSON shapes change.
Frontend	React TypeScript UI	Same React UI or near-same components	UI may call old paths or expect Laravel response shape.
Auth	Laravel Sanctum or API token pattern	JWT or API key/session style in FastAPI	Headers, 401 handling, and current user loaders can fail.
RAG/resources	Resource upload and retrieval logic	FastAPI upload, extraction, chunking	goes on. If missing metadata, fail silently or chunk poorly.
Voice	Admin TTS/STT endpoints existed	FastAPI equivalent routes	Audio MIME, file handling, and browser support can drift.

Important tone rule

When Sumon asks what is missing, the assistant should not give vague theory. It should produce a concrete gap list, ideally grouped by backend endpoint, frontend component, database/schema, auth, and testing steps.

Page 3 - TOMAA Product Context

TOMAA stands for a content automation product associated with Top Of Mind Automation. It helps businesses stay visible by turning source content into multi-platform social assets. The product is not related to the automotive company or toma.com. Preferred reference properties include tomaa.tech, aiautomates.tech/toma, aiautomates.tech/testpage-page, and the TOMA/TOMAA app dashboard paths.

Core promise

TOMAA automates omni-channel content creation from blogs, YouTube videos, topics, and launch ideas. It can summarize source material, create short summaries, generate video scripts, generate image concepts, produce optimized titles and descriptions, and prepare platform-specific content for Instagram, Facebook, LinkedIn, Threads, X, Reels, Shorts, TikTok, and other channels.

Business philosophy

The main operating belief is that consistent posting trains platform algorithms to recognize a brand, increase visibility, and build more touchpoints over time. TOMAA is not positioned as random AI posting. It is positioned as a system that takes real business knowledge and repackages it into daily content at scale while keeping human approval in the loop.

TOMAA concept	Meaning inside the system
Omni-channel content	One source can become many platform-specific posts, captions, scripts, and assets.
Human in the loop	AI drafts content, but a person can review, edit, approve, schedule, or publish.
30+ AI agents	A shorthand for specialized prompt/processing steps for summaries, visuals, captions, platform optimization, and repurposing.
Visibility engine	Repeated publishing creates thousands of monthly touchpoints and keeps the brand top of mind.
Time savings	The system is framed as saving roughly 30+ hours per month for teams that currently create content manually.

Platform examples

- Blog URL -> summary -> short summary -> social post pack -> image concept -> video script.
- YouTube URL -> extracted topic -> short-form script -> Reels/Shorts/TikTok copy.
- Launch topic -> launch announcement content -> LinkedIn, Facebook, email-style, and short video outputs.
- Existing post -> refreshed captions, alternate hooks, and platform-native descriptions.

Testing note

If a bot confuses TOMAA with a car company or only gives generic social media advice, it has failed project grounding. The correct answer should connect TOMAA to AI automation, content repurposing, multi-platform publishing, approval flow, and the business goal of staying top of mind.

Page 4 - Historical Laravel API Baseline

The earlier system used Laravel as the backend API. Laravel controllers handled authentication, content generation, customer settings, publishing, Blotato integration, prompt storage, media generation, and publish logs. The React UI expected certain response shapes, bearer-token headers, and route names. The FastAPI replacement should not simply expose new routes; it should preserve behavioral compatibility where the UI depends on it.

Important Laravel concepts to preserve

- ContentGeneration records store generated outputs for summary, short summary, video script, image prompt, social titles, and social content fields.
- PromptBlog or prompt-setting records define per-customer prompt templates.
- CustomerSetting stores service keys and social account IDs, including Blotato API settings.
- PublishController validates platform and post type, maps fields, uploads media to Blotato when needed, builds target payloads, and logs provider responses.
- PublishLog stores posted/failed/queued status, provider post IDs, final status, provider response, and sometimes public URLs.
- Frontend components poll publish status and latest logs to display progress.

Representative route behavior

POST /api/generate-contents

Body: { customer_id, url, reset? }

Expected behavior:

1. Validate customer_id and url.
2. Fetch article or source text.
3. Render prompt templates using context variables.
4. Generate outputs in sequence.
5. Update progress and status.
6. Return content_generation id and generated output fields.

POST /api/publish

Body: { content_generation_id, customer_id, platform, post_type, scheduled_at? }

Expected behavior:

1. Validate platform and post_type.
2. Enforce media rules for TikTok, YouTube Shorts, and Reels.
3. Resolve target accounts.
4. Upload media if not already hosted.
5. Send payload to provider.
6. Write PublishLog for every target.
7. Return per-target results.

Migration rule

The FastAPI backend should be judged against the behavior that the React UI already expects, not only against abstract REST conventions. If the UI calls /api/content-generations/:id or /api/publish/logs/latest, then either FastAPI must implement those paths or the UI must be updated consistently. The same applies to auth headers, JSON keys, status naming, pagination, and error messages.

Page 5 - FastAPI Target Architecture

The FastAPI target should separate concerns clearly. Routers should stay thin. Services should contain business logic. Pydantic schemas should define request and response models. Database models should map the persistent structure. Background jobs or task workers should handle long-running generation and media tasks when needed. The chatbot API should be reliable under long documents, multiple resources, guided flows, and repeated user sessions.

Suggested module structure

```
app/  
  main.py  
  core/  
    config.py  
    security.py  
    database.py  
  models/  
    user.py  
    customer.py  
    content_generation.py  
    resource.py  
    conversation.py  
    publish_log.py  
  schemas/  
    auth.py  
    chat.py  
    resource.py  
    content_generation.py  
    publish.py  
  routers/  
    auth.py  
    chat.py  
    resources.py  
    content_generation.py  
    publish.py  
    tts.py  
    stt.py  
  services/  
    rag_service.py  
    embedding_service.py  
    document_parser.py  
    chat_service.py  
    publish_service.py  
    blotato_service.py  
    prompt_service.py  
  workers/  
    generation_tasks.py  
    media_tasks.py
```

FastAPI implementation priorities

- Preserve frontend compatibility before redesigning API names.
- Use Pydantic models for strict validation and predictable errors.
- Support PDF upload, extraction, chunking, embedding, and retrieval with useful metadata.
- Track document processing status so the UI knows whether a resource is ready.
- Store chunk source references so RAG answers can cite the right resource.
- Keep guided-flow state separate from freeform chat state.
- Use dependency injection for current user, database session, and API key auth.
- Add structured logs around extraction, embedding, retrieval, model call, and response.

Layer	Responsibility	Bad smell
Router	HTTP input/output only	Large prompt logic inside endpoint function.
Service	Business logic and orchestration	Directly reading request objects everywhere.
Repository/model	Database access and persistence	Embedding calls mixed into ORM model methods.
Schema	Validation and serialization	Returning raw database objects with hidden fields.
Worker	Long-running tasks	Blocking upload request until all embeddings finish.

Page 6 - React TypeScript UI Compatibility

The UI should remain the same as before where possible. Existing React components may include chat widgets, standalone preview pages, customer dashboards, publish pages, content generation forms, status polling, audio playback, speech-to-text, text-to-speech, and localStorage-based session keys. When replacing Laravel with FastAPI, the biggest risk is that the UI silently expects old response fields.

Known UI concepts from chatbotDev

- src/lib/chatApi.ts includes postChat, postTts, and postStt helpers.
- ChatPayload includes optional visitor data.
- ChatWidget.tsx handles conversation state, localStorage keys, guided flow, audio controls, and composer UI.
- WidgetStandalone.tsx is used for previewing the widget in a phone-frame style layout.
- The widget supports browser STT/TTS detection with MediaRecorder and server fallbacks.
- The UI may preserve conversation state by apiKey plus externalUserId hash.
- Per-message audio playback and auto-read replies can exist as toggles.

Compatibility checklist

UI expectation	FastAPI must provide
POST chat endpoint returns assistant message	Stable JSON with answer/message, conversation id, sources, and optional flow state.
Resource upload shows progress or readiness	Upload endpoint plus status endpoint or synchronous result for small files.
STT endpoint accepts audio blob	multipart/form-data with correct field name and MIME handling.
TTS endpoint returns playable audio	audio/mpeg or configured content-type with no JSON wrapping unless UI expects it.
Guided flow continues across turns	server-side state or client-provided state accepted and returned.
Errors display in UI	clear status codes and message fields, not generic server traces.

Important front-end migration rule

Do not update only the backend and assume the UI will work. Search for every API helper and every fetch call. Check base URL, headers, token key, request body shape, response body shape, and error parser. The minimum test is: login or API-key handshake, start chat, send message, upload resource, ask question from resource, use guided flow, test TTS, test STT, refresh page, continue conversation.

Page 7 - RAG, PDF Processing, and Long Resource Testing

This document itself is intended as a long resource ingestion test. A strong bot should parse all pages, chunk content without losing headings, embed useful chunks, retrieve the right sections for questions, and avoid mixing unrelated projects. Long PDF ingestion should not fail just because the file has tables, pseudo-code, bullets, platform names, repeated words, and overlapping concepts.

Resource processing pipeline

```
Upload PDF -> store file metadata -> extract text per page -> normalize whitespace
-> preserve page numbers and headings -> chunk by section and token budget
-> embed chunks -> store vectors and source metadata -> mark resource as ready
-> retrieve top chunks for a user question -> build grounded prompt
-> answer with citations or source names -> optionally include confidence/debug data.
```

Chunking guidelines

- Keep page number, heading, resource id, and chunk index on every chunk.
- Do not chunk only by fixed character length; respect headings where possible.
- Use overlap, but not so much that retrieval returns near-duplicates only.
- Avoid stripping important domain terms like TOMAA, Blotato, FastAPI, React TypeScript, Laravel, guided flow, and Sumon.
- When the PDF includes tables, convert rows into text that preserves column meaning.
- When extraction fails or returns too little text, mark the resource as failed and expose the reason.

Questions the new bot should answer from this PDF

- Who is Sumon in this project context?
- What is TOMAA and what is it not?
- Why is FastAPI migration risky if the React UI remains unchanged?
- Which Laravel behaviors should be preserved?
- What are the main RAG pipeline steps for long PDF ingestion?
- What should happen when a PDF extraction fails?
- What does human-in-the-loop mean for TOMAA?
- What UI features need compatibility testing after backend migration?

Failure modes

Bad retrieval will confuse Jimmy and Sumon, treat TOMAA as a generic brand, hallucinate unrelated routes, ignore the Laravel baseline, or answer from memory instead of this resource. Good retrieval will quote or cite the relevant section, explain only what the document supports, and state uncertainty when a detail is not inside the uploaded resource.

Page 8 - Guided Flow and Chatbot Behavior Requirements

The chatbot should support both freeform RAG chat and guided flows. A guided flow is not just a normal chat response. It collects structured information step by step, can ask follow-up questions, can validate fields, and can eventually produce a lead, ticket, booking request, support request, qualification record, or another structured payload. The frontend may display the flow as normal messages, but the backend needs state.

Guided-flow concepts

Concept	Description
Flow definition	A named set of steps, expected fields, validation rules, and completion action.
Flow state	Current step, collected values, pending validation, and completion flag.
Visitor object	Optional user/visitor metadata passed from the widget or host site.
Session key	Stable identifier for continued conversations across page refresh.
Fallback mode	When no flow is active, answer normally using RAG and system rules.
Handoff/action	When the flow completes, create a ticket, lead, notification, or API payload.

Example chat request shape

```
{
  "message": "I need help setting up TOMAA for my business",
  "conversation_id": "optional-existing-id",
  "api_key": "site-widget-key",
  "external_user_id": "optional-host-user-id",
  "visitor": {
    "name": "Test Visitor",
    "email": "visitor@example.com",
    "source_url": "https://example.com/tomaa-page"
  },
  "flow_state": {
    "active_flow": "tomaa_lead_qualification",
    "current_step": "business_goal",
    "values": { "company": "Example Co" }
  }
}
```

Expected chat response behavior

- Return the assistant reply in a stable field the UI can render.
- Return updated flow_state when a guided flow is active.
- Return sources when the answer used uploaded resource context.
- Do not expose internal chain-of-thought or raw prompt assembly.
- Use validation messages that are user-friendly but precise.
- Let normal RAG answers and guided flow questions coexist without breaking the UI.

Page 9 - TOMAA Data, Publishing, and Platform Details

TOMAA content generation and publishing revolve around source content, generated outputs, media URLs, platform-specific copy, approval, scheduling, and final posting logs. Even if chatbotDev is not the TOMA app itself, TOMAA is a major reference domain for testing whether the bot can handle project-specific business and technical details.

Representative content-generation fields

Field group	Examples
Source	url, source_type, source_title, source_text, customer_id
Core outputs	summary, short_summary, video_script, image_concept
Facebook	facebook_title, facebook_content, facebook_reels_title, facebook_reels_content
Instagram	instagram_title, instagram_content, instagram_reels_title, instagram_reels_content
LinkedIn	linkedin_personal_title, linkedin_personal_content, linkedin_page_title, linkedin_page_content
Short video	tiktok_title, tiktok_content, youtube_short_title, youtube_short_content
Media	image_url, video_url, media_upload_status, generated_asset_metadata
Status	status, progress, error_message, posted_on, public_url

Publishing rules

- TikTok and YouTube Shorts generally require video post type.
- Reels flows require video media, not just text.
- X/Twitter content should be trimmed to fit platform limits when required.
- Scheduled posts should convert local UI datetime into UTC ISO before sending to the API.
- Provider responses should be logged so the UI can show whether the post is queued, posted, failed, or still processing.
- A public URL should be displayed only when the provider returns or exposes one.

Example publish payload

```
{
  "content_generation_id": 123,
  "customer_id": 1,
  "platform": "linkedin_page",
  "post_type": "image",
  "scheduled_at": "2026-06-05T15:00:00Z",
  "title": "How TOMAA Keeps Your Brand Top of Mind",
  "content": "Turn one blog or video into a full month of visibility...",
  "image_url": "https://cdn.example.com/tomaa/generated-image.png"
}
```

Bot testing instruction

If asked how TOMAA publishes content, the bot should mention source content, generated outputs, approval/scheduling, platform-specific fields, media requirements, provider posting, and logs. It should not answer only with generic social media best practices.

Page 10 - Acceptance Tests for the New FastAPI Bot

Use this page as a final test plan after uploading the PDF into the new bot. The bot should answer based on the uploaded resource, not broad internet knowledge. The most important test is whether it can keep Sumon, Jimmy, TOMAA, chatbotDev, Laravel, React TypeScript, and FastAPI separate but connected.

Retrieval tests

Question	Expected grounded answer
Who is Sumon?	The developer/programmer persona working on chatbotDev and related projects, especially technical implementation.
What is TOMAA?	An omni-channel AI content automation product focused on repurposing blogs, YouTube videos, topics, and launch ideas into platform-ready content.
What is TOMAA not?	It is not the automotive company at toma.com.
Why preserve Laravel behavior?	Because the React TypeScript UI may depend on old route names, response shapes, statuses, and auth handling.
What should the PDF pipeline store?	Resource metadata, page numbers, headings, chunks, embeddings, and source references.
What must TTS return?	Playable audio with correct MIME/content-type if the UI expects direct audio.

Implementation acceptance criteria

- FastAPI routes match or intentionally replace all UI-called Laravel routes.
- React API helpers are updated in one place and use consistent base URLs and headers.
- Long PDF upload processes successfully and returns ready/failed status.
- RAG answers cite or reference the uploaded resource sections.
- Guided flow state survives multi-turn conversations.
- TTS and STT are tested from the real browser widget, not only with Postman.
- Errors are visible and actionable in both backend logs and UI messages.
- The system avoids hallucinating unsupported details when the resource lacks the answer.

Final note for the bot under test

When answering questions from this document, stay grounded. Sumon wants implementation clarity. Jimmy represents the business owner side. TOMAA is the content automation product. chatbotDev is the current chatbot development project. Laravel is the old backend baseline. FastAPI is the new backend target. React TypeScript UI compatibility is critical. Long complex PDF processing is the reason this resource exists.

End of 10-page resource pack.